

Reverse Engineering Workshop

10/12/2025

At ENSI, 2PM

I. Concepts and Foundations

1. Hardware Foundations

Reverse engineering starts with understanding **how a computer executes instructions**.

CPU (Central Processing Unit)

The CPU is the component responsible for **executing instructions**.

Registers:

Very small, **extremely fast memory** locations **inside the CPU**.

All operations (addition, subtraction, comparison, etc.) occur inside registers.

Examples: **EAX, EBX, ECX, ESP, RIP**.

RAM (Random Access Memory)

Temporary, **volatile memory where programs store data during execution**.

Slower than CPU registers but much faster than storage devices (HDD/SSD).

Holds program variables, instructions being processed, and runtime data.

2. From Binary to Human Language

Originally, computers understand **only binary (0s and 1s)**.

To allow humans to go from binary to human language, **programming languages** were developed.

Programming Languages Have 3 Levels

A. Low-Level Languages

Very close to machine code.

Hard to read, but give full control over hardware.

Example: **Assembly** language.

Each instruction maps almost directly to binary machine instructions.

Used in reverse engineering because it reveals exactly what the program does.

B. Middle-Level Languages

Combine low-level control with higher-level readability.

Example: C language.

Closer to human language but still maps efficiently to machine instructions.

Widely used for systems programming.

C. High-Level Languages

Farther from hardware, more human-friendly syntax.

Examples: Python, JavaScript.

Often use **interpreters** or **virtual machines**.

Not directly tied to CPU instructions.

3. From Human Code to Machine Code

To convert **human-written programs into binary**, we need translation tools:

Compilers

Take high-level languages (C, C++, Rust...)

Convert them into machine code (binary).

Perform optimizations, checks, and produce an executable file.

Assemblers

Take assembly instructions (MOV, ADD, SUB...)

Translate them directly to machine code.

Interpreters

Read and execute high-level code line by line.

Do not usually produce standalone binaries.

4. Each Binary Sequence Has a Meaning

Every sequence of bits (0 and 1) corresponds to:

An instruction

A memory address

A value

A flag

An operation

The CPU interprets these binary patterns according to the architecture (x86, x64, ARM).

5. Common Assembly Instructions

These are frequently encountered during reverse engineering:

MOV: Move data between registers, memory, or immediate values.

Example: **MOV EAX, 5** → load 5 into register EAX.

ADD: Perform addition.

Example: **ADD EAX, EBX** → $EAX = EAX + EBX$.

SUB: Perform subtraction.

Example: **SUB ECX, 2** → $ECX = ECX - 2$.

SYSCALL:

Triggers a system call to request an OS-level service.

Example: reading a file, allocating memory, printing output.

Transfers execution from user space to kernel space.

These instructions represent the core logic of executables at the lowest level.

6. Reversing Tools: Decompiler & Disassembler

Disassembler

Converts machine code → **assembly**.

Shows the **actual CPU instructions**.

Example tools: IDA Pro, Ghidra, radare2.

Decompiler

Converts machine code → **pseudo-C** or another high-level approximation.

Does **not recover original source code**, but **provides a readable structure**.

Helps understand logic (loops, conditions, variables) more easily.

Pseudo-C

A “guess” by the decompiler.

Reconstructs: Functions, Control flow, Variables (when possible), Loops (**for**, **while**), Conditions (**if**, **else**)

Useful for understanding functionality, but **not exact**.

Many reverse engineers switch between:

Assembly view → for exact behavior

Pseudo-C view → for readability

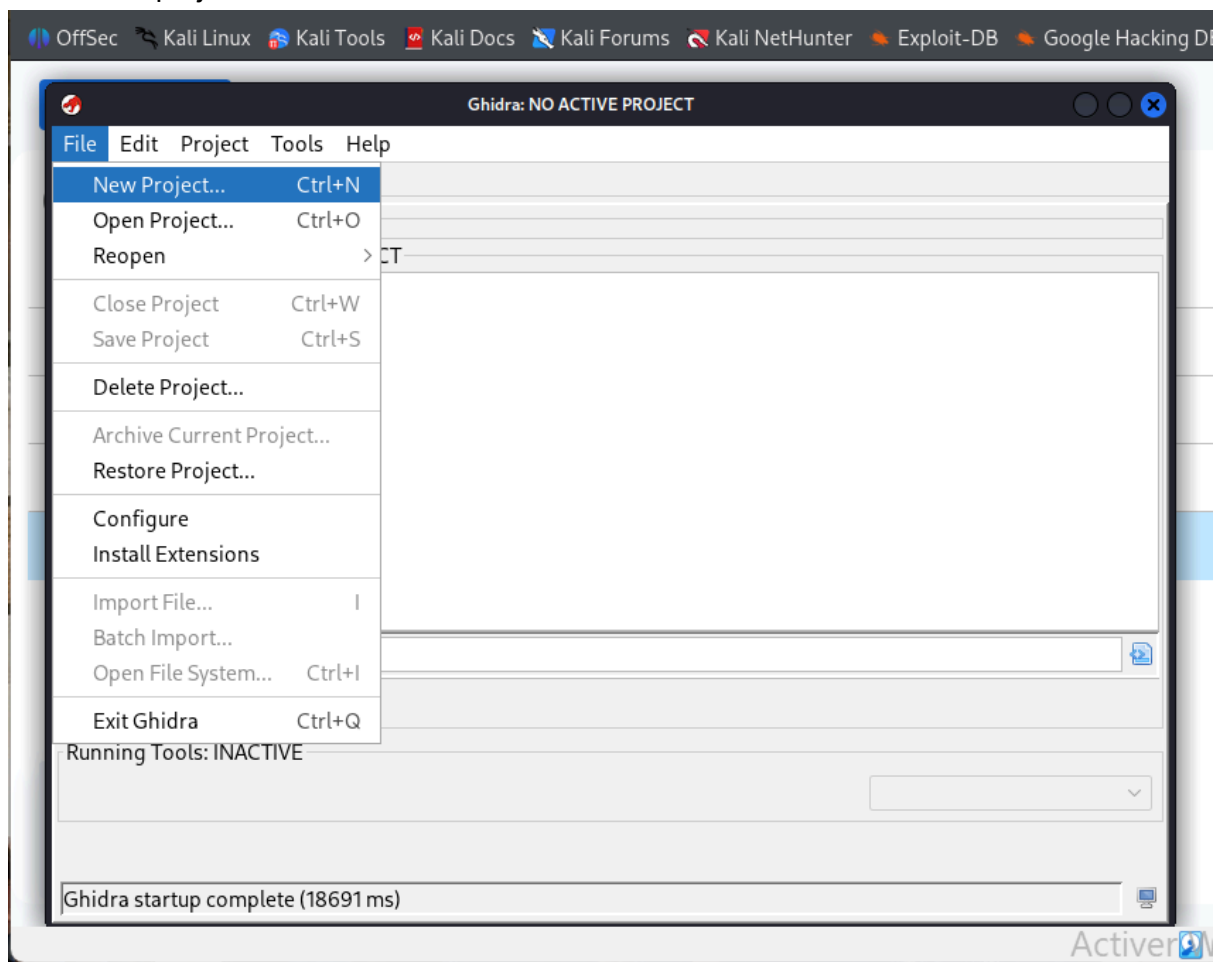
II. Practise

1. Install ghidra

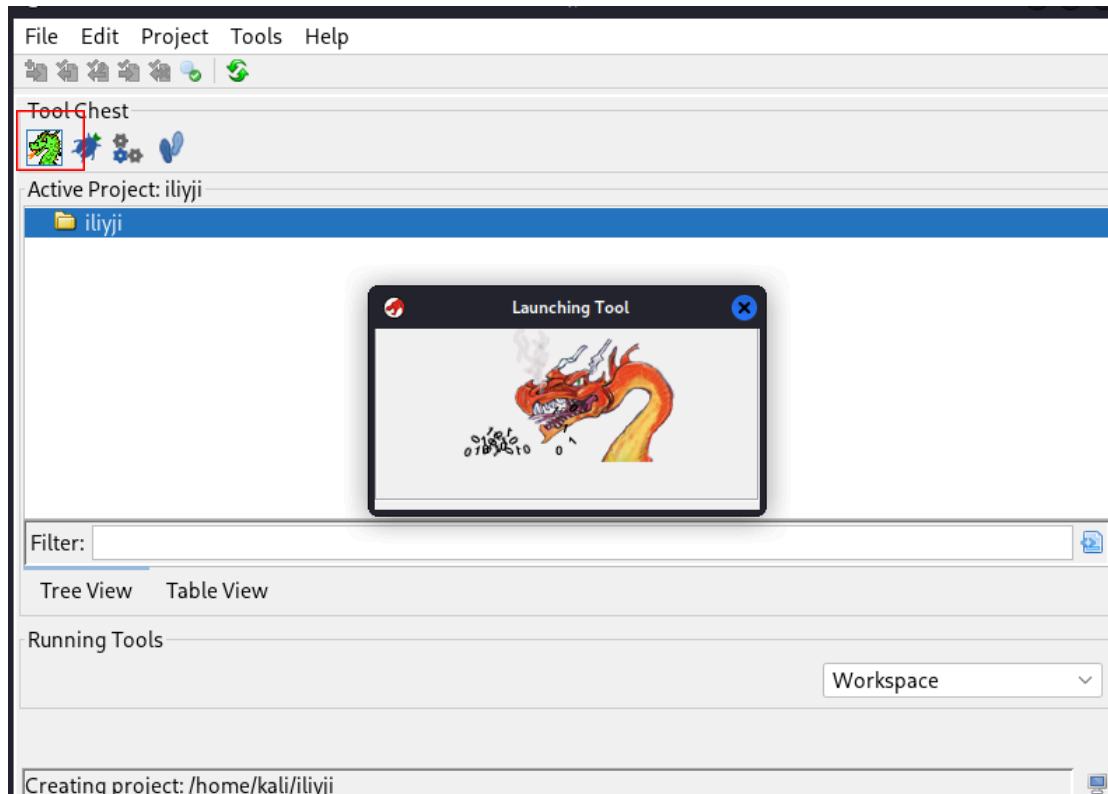
Sudo apt install ghidra

2. Open Ghidra

Select new project & name it



Click on launching tool



Download the data from this drive:

<https://drive.google.com/drive/mobile/folders/1nkTvDE-h3oaUUIZVFuLSAbZbSik4iA-A?usp=sharing>

Unzip file

```
(kali㉿kali)-[~/Downloads]
$ ls
DarkwebForum.pcap  digdan2  readme.txt  tasks1-5.zip
digdan             fogged.bin  runme

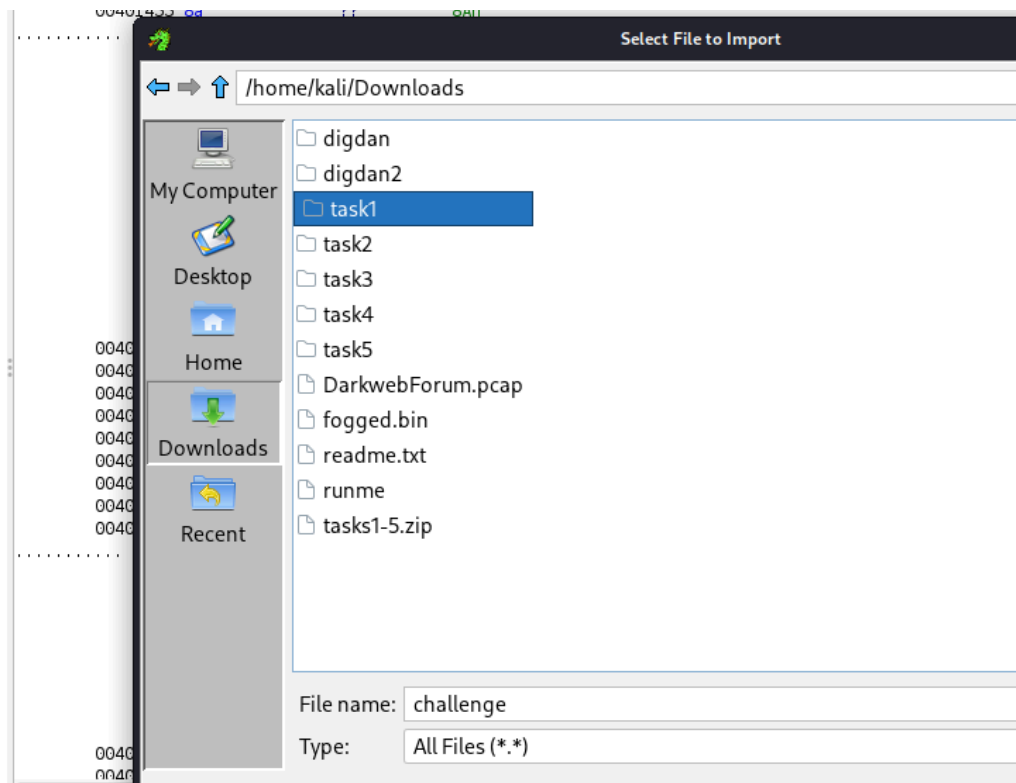
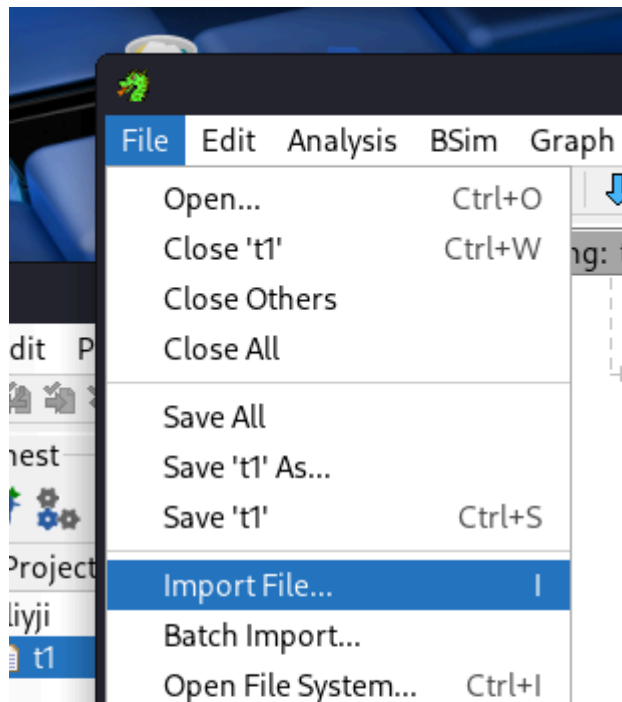
(kali㉿kali)-[~/Downloads]
$

(kali㉿kali)-[~/Downloads]
$ unzip tasks1-5.zip

Archive:  tasks1-5.zip
  creating: task1/
  inflating: task1/t1
  creating: task2/
  inflating: task2/t2
  creating: task3/
  inflating: task3/t3
  creating: task4/
  inflating: task4/t4
  creating: task5/
  inflating: task5/portals.bin
  inflating: task5/t5
```

3. Task1

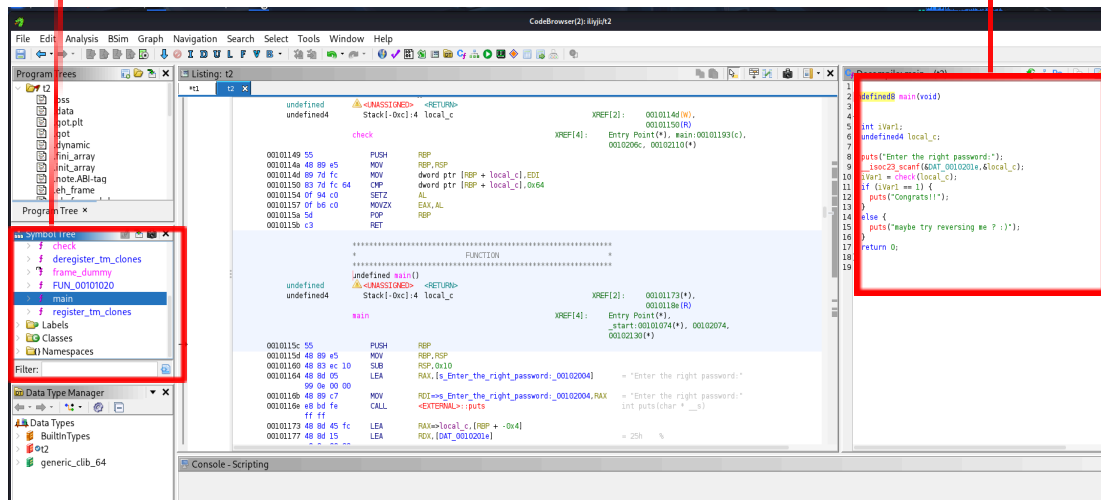
Import task1



Click on analyse then ok

Functions that exists

Reading functions



Read and understand the code in main function

```

Decompile: main - (t1)
1
2 undefined8 main(void)
3
4 {
5     int iVar1;
6     char local_13 [11];
7
8     puts("Enter the right password:");
9     __isoc23_scanf(&DAT_0010201e,local_13);
10    iVar1 = strcmp(local_13,"Securinets");
11    if (iVar1 == 0) {
12        puts("Congrats!!");
13    }
14    else {
15        puts("maybe try reversing me ? :)");
16    }
17    return 0;
18 }
19

```

```

(kali㉿kali)-[~/Downloads]
$ ./task1

(kali㉿kali)-[~/Downloads/task1]
$ ./t1
zsh: permission denied: ./t1

(kali㉿kali)-[~/Downloads/task1]
$ chmod +x t1

(kali㉿kali)-[~/Downloads/task1]
$ ./t1
Enter the right password:

```

We use chmod to add the execution permission (look in linux cours)
This is to understand more

```

(kali㉿kali)-[~/Downloads/task1]
$ exiftool t1
ExifTool Version Number      : 13.36
File Name                    : t1
Directory                    : .
File Size                    : 16 kB
File Modification Date/Time   : 2025:12:09 18:02:42-05:00
File Access Date/Time        : 2025:12:10 08:50:14-05:00
File Inode Change Date/Time   : 2025:12:10 08:50:12-05:00
File Permissions              : -rwxrwxrwx
File Type                    : ELF shared library
File Type Extension          : so
MIME Type                    : application/octet-stream
CPU Architecture              : 64 bit
CPU Byte Order                : Little endian
Object File Type              : Shared object file
CPU Type                      : AMD x86-64

```

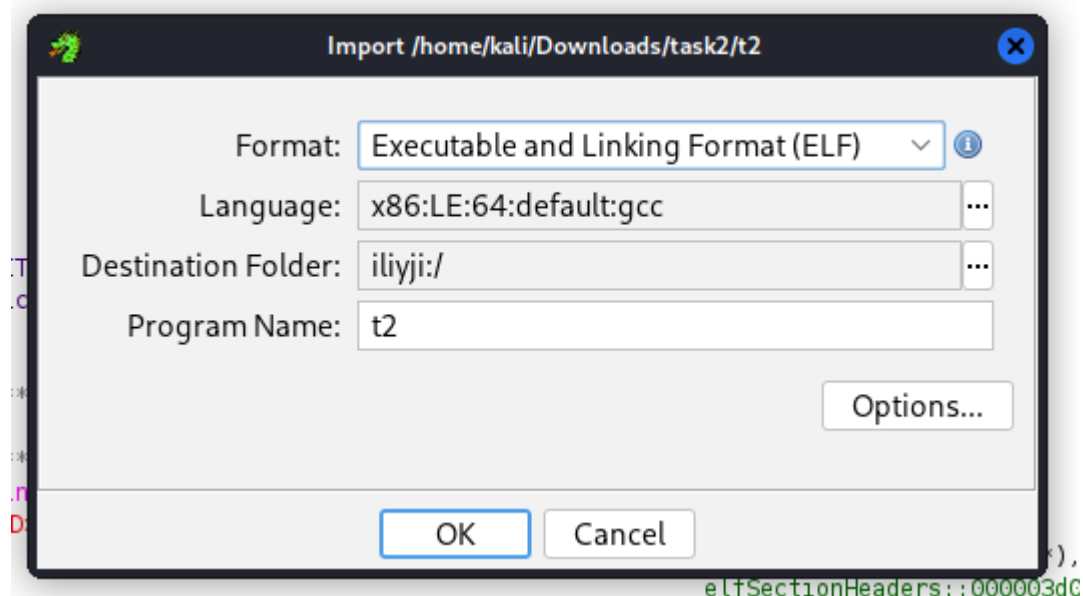
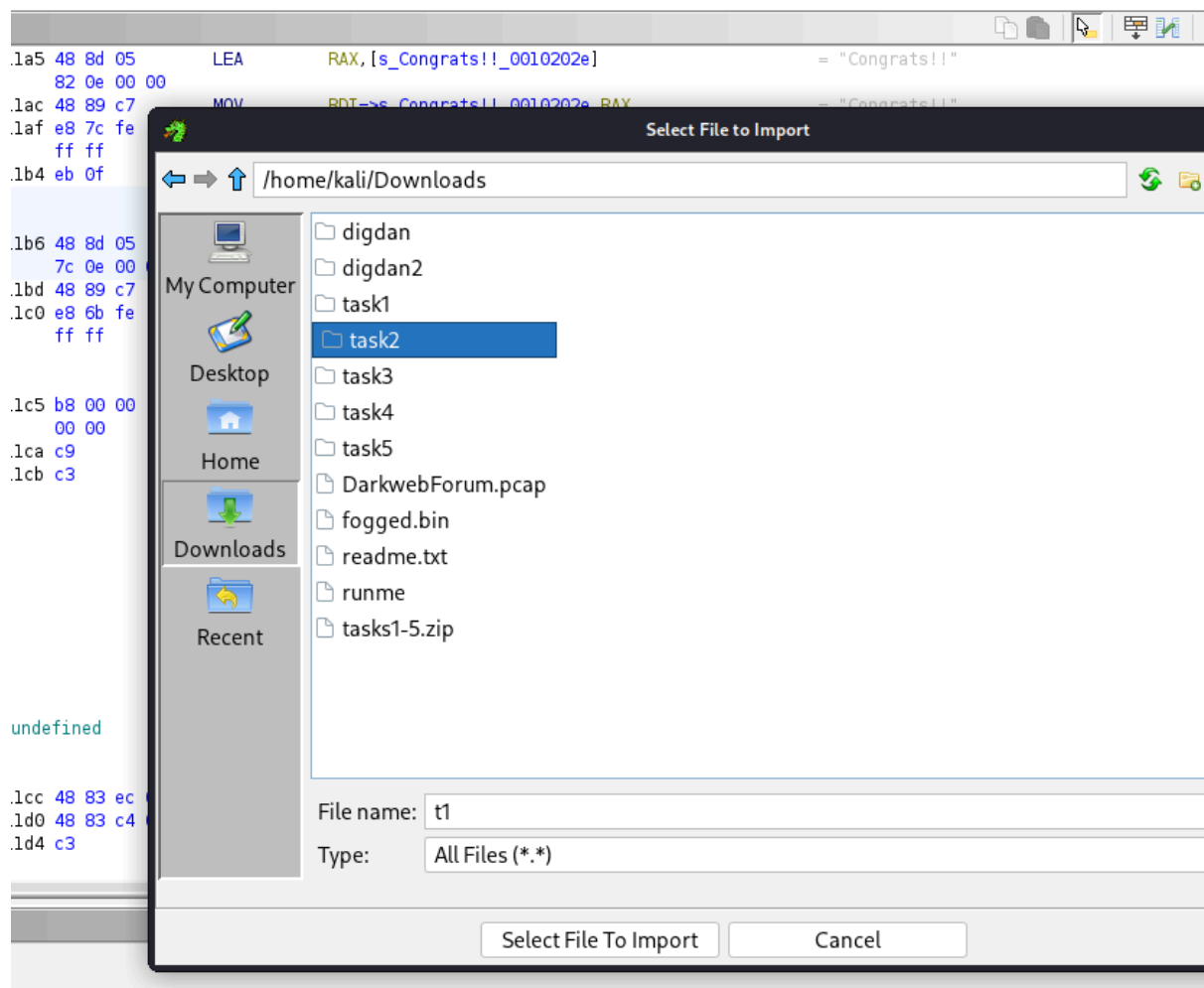
```

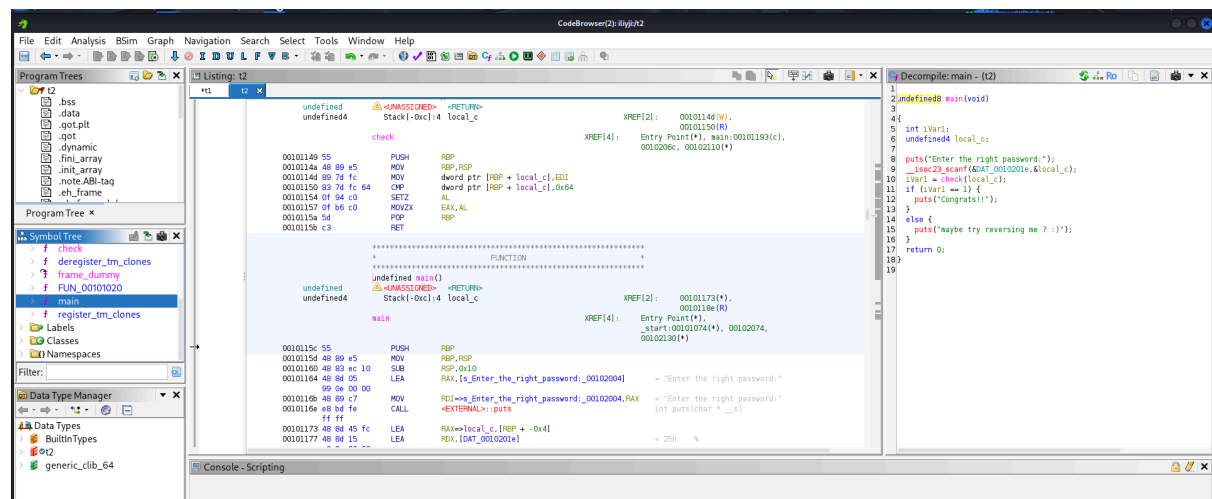
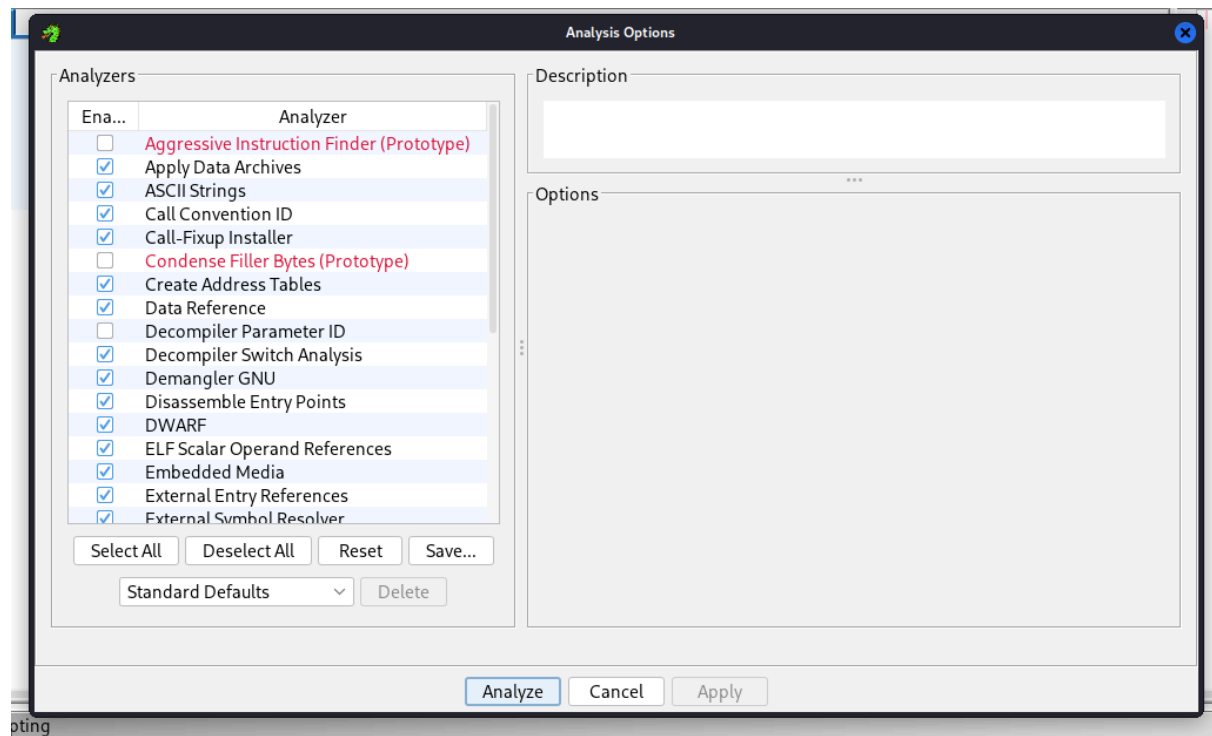
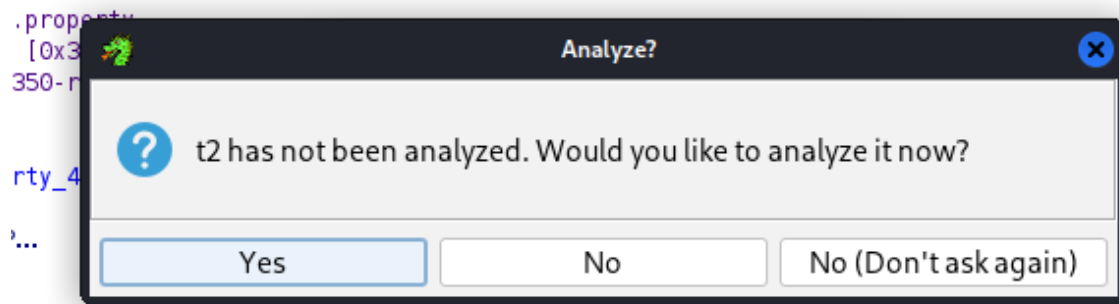
(kali㉿kali)-[~/Downloads/task1]
$ ls
t1

(kali㉿kali)-[~/Downloads/task1]
$ ./t1
Enter the right password:
Securinets
Congrats !!

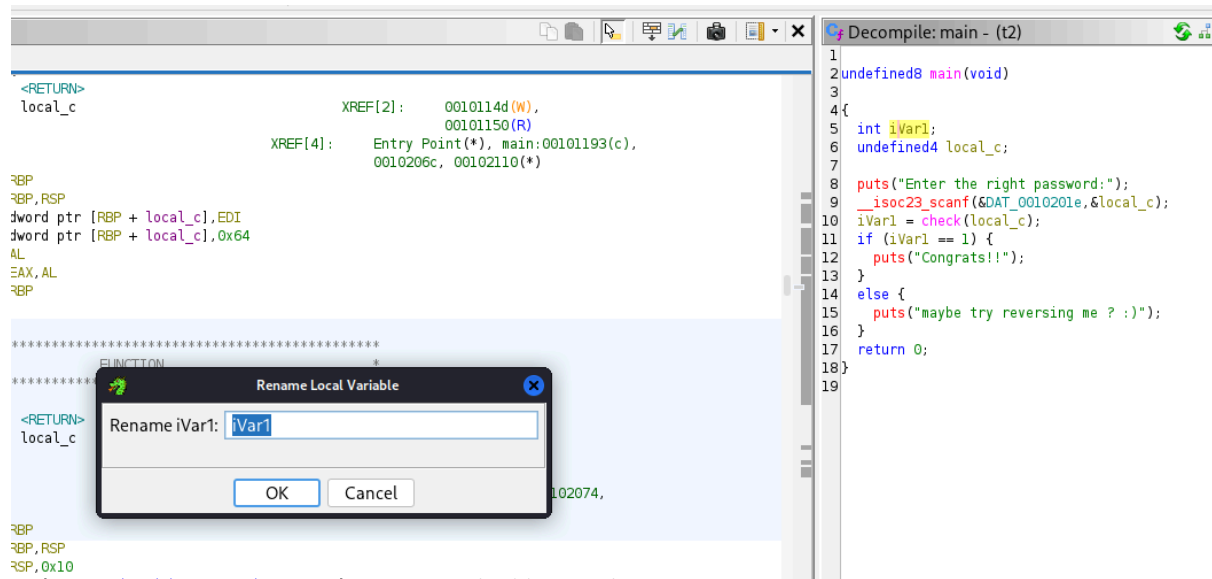
```


4. Task 2

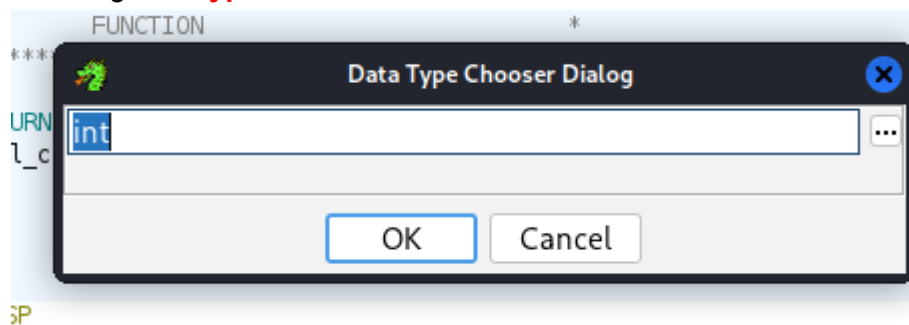




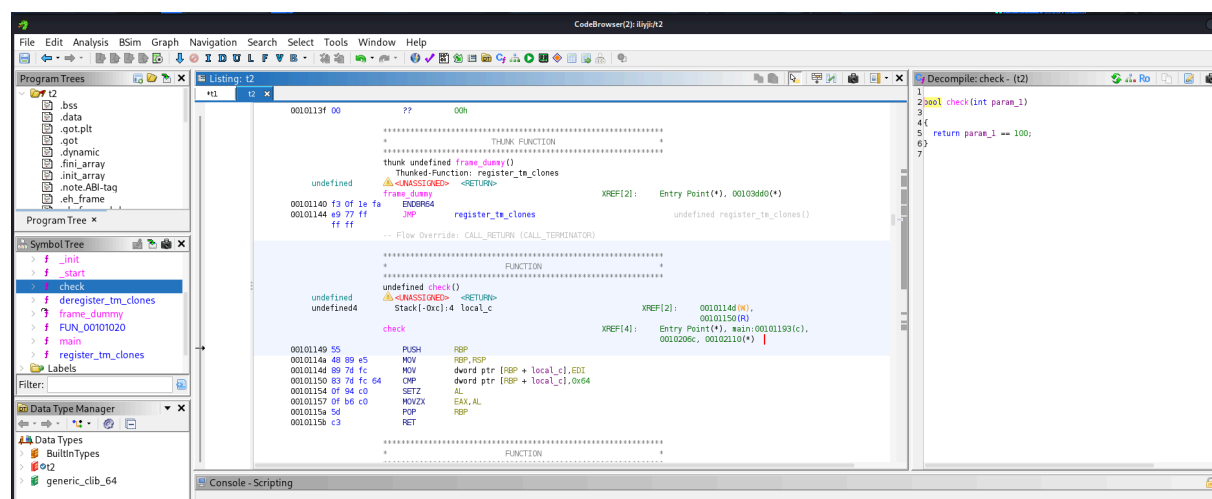
To change **name** of the variable: select variable then click on **L**



To change the **type** of the variable: select variable then **CTRL + L**



Understand the code



_isoc23_scanf : That's for the input

```

(kali@kali)-[~/Downloads/task1]
$ ..

(kali@kali)-[~/Downloads]
$ ./task2

(kali@kali)-[~/Downloads/task2]
$ ls
t2

(kali@kali)-[~/Downloads/task2]
$ ./t2
zsh: permission denied: ./t2

(kali@kali)-[~/Downloads/task2]
$ chmod +x t2

(kali@kali)-[~/Downloads/task2]
$ ./t2
Enter the right password:
100
Congrats !!

```

5. Task 3

Import data

Understand code

```

1
2 bool check_prefix(char *param_1)
3
4 {
5     int iVar1;
6
7     iVar1 = strncmp(param_1,"Securinets{",0xb);
8     return iVar1 == 0;
9 }
10

```

,0xb);

Hex Decimal		
byte	Bh	11
char	'\\v'	

this means your input

```

2 undefined8 check_inner(char *param_1)
3
4 {
5     size_t sVar1;
6     undefined8 uVar2;
7     char local_19 [9];
8     int local_10;
9     int local_c;
10
11     builtin_strncpy(local_19,"c0ngr4tz",8);
12     sVar1 = strlen(param_1);
13     local_10 = (int)sVar1 + -0xc;
14     if (local_10 == 8) {
15         for (local_c = 0; local_c < 8; local_c = local_c + 1) {
16             local_19[8] = param_1[local_c + 0xb];
17             if (param_1[local_c + 0xb] != local_19[local_c]) {
18                 return 0;
19             }
20         }
21         uVar2 = 1;
22     }
23     else {
24         uVar2 = 0;
25     }
26     return uVar2;
27 }

```

Input

```

Decompile: check_suffix - (t3)
1
2 bool check_suffix(char *param_1)
3
4 {
5     size_t sVar1;
6
7     sVar1 = strlen(param_1);
8     return param_1[(long)(int)sVar1 + -1] == '}' ;
9 }

```

```

(kali㉿kali)-[~/Downloads/task2]
$ ..

(kali㉿kali)-[~/Downloads]
$ cd task3

(kali㉿kali)-[~/Downloads/task3]
$ ./t3
zsh: permission denied: ./t3

(kali㉿kali)-[~/Downloads/task3]
$ chmod +x t3

(kali㉿kali)-[~/Downloads/task3]
$ ./t3
Enter the flag:
Securinets{c0ngr4tz}
Congrats!

```

6. Task 4

What is the Stack?

The **stack** is a **special region of memory** used **by programs** to **store**:

Local variables, Function parameters, Return addresses, Saved registers, Temporary data

How the Stack Works?

You can imagine the stack as a **vertical bucket** or a **pile of plates**:

You add new items on top.

You remove items only from the top.

This is called **LIFO: Last In, First Out**.

PUSH and POP Instructions

PUSH

Saves a value onto the top of the stack.

Moves **RSP** **down** (because the stack grows down in memory).

Example:

```
PUSH RAX
```

This decreases **RSP** and puts the value of RAX on the stack.

POP

Removes (loads) the last value from the top of the stack.

Moves **RSP** **up**.

Example:

```
POP RBX
```

This takes the top value from the stack and stores it in RBX.

What is an Offset?

An **offset** is simply the **distance (in bytes)** between one memory location and another.

In assembly, we often access local variables using offsets relative to **RBP**.

Example:

```
mov rax, [rbp - 8]
```

Meaning:

-8: offset

The offset is a **byte distance** from the base pointer.

It is a distance between a **register (RBP)** and a **memory location**.

Go to the memory address **8 bytes below RBP**

Load that value into **RAX**

Why negative offsets?

When a function starts, it creates a **stack frame**.

Local variables are stored *below* RBP → negative offsets.

Why positive offsets?

Function parameters passed on the stack (in some calling conventions) are stored *above* RBP → positive offsets.

Import task 4

Read code

```
00101139 55          PUSH      RBP
0010113a 48 89 e5     MOV       RBP,RSP
0010113d 48 83 ec 10   SUB       RSP,0x10
00101141 c7 45 fc     MOV       dword ptr [RBP + local_c],0x0
00101148 eb 04        JMP       LAB_0010114e

LAB_0010114a
0010114a 83 45 fc 01   ADD       dword ptr [RBP + local_c],0x1
XREF[1]: 00101152(j)

LAB_0010114e
0010114e 83 7d fc 04   CMP       dword ptr [RBP + local_c],0x4
XREF[1]: 00101148(j)
00101152 7e f6        JLE       LAB_0010114a
00101154 83 7d fc 05   CMP       dword ptr [RBP + local_c],0x5
00101158 75 16        JNZ       LAB_00101170
0010115a 48 8d 05     LEA       RAX,[s_ERROR.....The_program_has_b_00102... = "ERROR.....The program h...
a7 0e 00 00
00101161 48 89 c7     MOV       RDI=>s_ERROR.....The_program_has_b_00102... = "ERROR.....The program h...
00101164 e8 c7 fe     CALL     <EXTERNAL>::puts
int puts(char * __s)
ff ff
00101169 b8 01 00     MOV       EAX,0x1
00 00
0010116e eb 14        JMP       LAB_00101184

LAB_00101170
00101170 48 8d 05     LEA       RAX,[s_Make_the_binary_print_me:)_00102038]
XREF[1]: 00101158(j)
c1 0e 00 00
00101177 48 89 c7     MOV       RDI=>s_Make_the_binary_print_me:)_00102038,RAX = "Make the binary print me :)"
0010117a e8 b1 fe     CALL     <EXTERNAL>::puts
int puts(char * __s)
ff ff
0010117f b8 00 00     MOV       EAX,0x0
00 00
```

To change **CTRL + Shift + G**

```
LAB_0010114a
0010114a 83 45 fc 01   ADD       dword ptr [RBP + local_c],0x1
XREF[1]: 00101152(j)

LAB_0010114e
0010114e 83 7d fc 04   CMP       dword ptr [RBP + local_c],0x4
XREF[1]: 00101148(j)
00101152 7e f6        JLE       LAB_0010114a
00101154 83 7d fc 05   CMP       dword ptr [RBP + local_c],0x5
00101158 eb 16        JMP       0x00101170
0010115a 48 8d 05     LEA       RAX,[s_ERROR.....The_program_has_b_00102... = "ERROR.....The program h...
a7 0e 00 00
00101161 48 89 c7     MOV       RDI=>s_ERROR.....The_program_has_b_00102... = "ERROR.....The program h...
00101164 e8 c7 fe     CALL     <EXTERNAL>::puts
int puts(char * __s)
ff ff
00101169 b8 01 00     MOV       EAX,0x1
00 00
0010116e eb 14        JMP       LAB_00101184

LAB_00101170
00101170 48 8d 05     LEA       RAX,[s_Make_the_binary_print_me:)_00102038]
XREF[1]: 00101158(j)
-- -- -- --
= "Make the binary print me :)"
```

Replacing **jnz** with **jmp** removes the condition linked to the Zero Flag and forces execution to always take a specific branch. This is commonly used in reverse engineering to bypass checks, reach protected code paths, or simplify program logic for analysis.

Detailed explanation:

What Are Jumps in Assembly?

Assembly uses *jump instructions* to **change the flow of execution**.

There are two categories:

A. Unconditional Jump

jmp

Always jumps to the target address.

No condition, no check.

Code flow is forced.

B. Conditional Jump

Based on CPU flags (**ZF**, **CF**, **OF**, etc.).

Examples:

jnz = jump if not zero

jz = jump if zero

je = jump if equal

jne = jump if not equal

jg, **jle**, **ja**, etc.

What Does **jnz** Mean?

jnz = **Jump if Not Zero**

It checks the **Zero Flag (ZF)** in the CPU status register.

ZF = 1 → **result is zero**

→ **jnz** does **NOT** jump

ZF = 0 → **result is NOT zero**

→ **jnz** jumps

So:

jnz target = "If the previous instruction produced a non-zero result, jump to target."

What Does jmp Mean?

`jmp target` = “Jump to target ALWAYS, regardless of flags.”

No condition.

No test.

No comparison.

Just jump.

What Happens When You Replace jnz With jmp?

You remove the condition.

Before:

```
cmp eax, 0
jnz valid
```

Flow:

If `eax != 0` → go to `valid`

If `eax == 0` → continue normally

After modification:

```
cmp eax, 0
jmp valid
```

Flow:

It **always** goes to `valid`, even if `eax` is 0.

You are bypassing the check.

Why Is This Done in Reverse Engineering Tasks?

There are **three main reasons**, depending on the challenge.

1. To Bypass Security Checks

Examples: Bypassing password verification, Skipping license validation, Breaking anti-debug or anti-tamper logic, Going directly to the “success” or “valid user” branch

Example:

```
cmp eax, 1
jnz fail    ; if wrong → fail
...
fail:
```

If you change to:

```
jmp success
```

Or more subtly:

```
jnz success → replace with jmp success
```

You make the program **always think the check passed**.

2. To Force Program Flow in CTF Challenges

In CTF reverse engineering tasks, authors often put a condition to block the player.

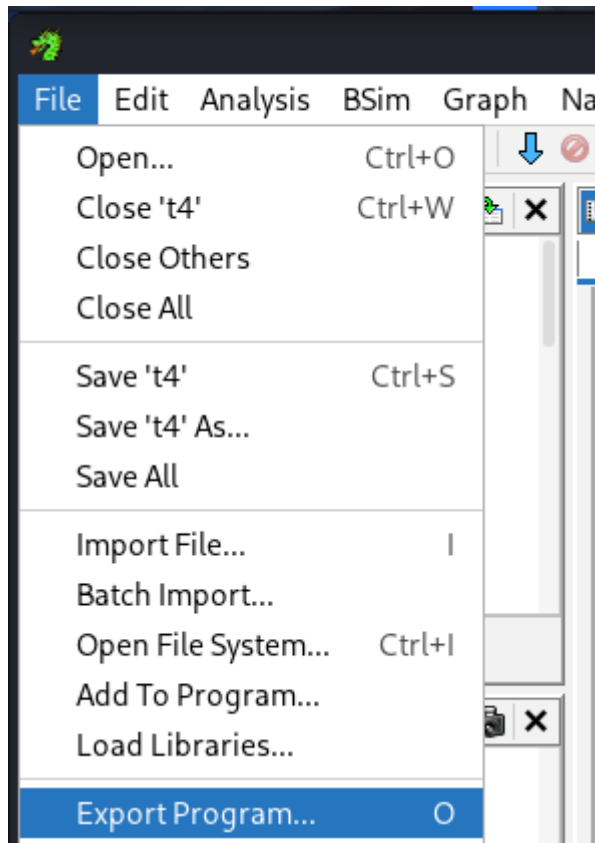
Changing a conditional jump → unconditional jump lets you reach:

the flag, the decryption code, the hidden function, the correct branch

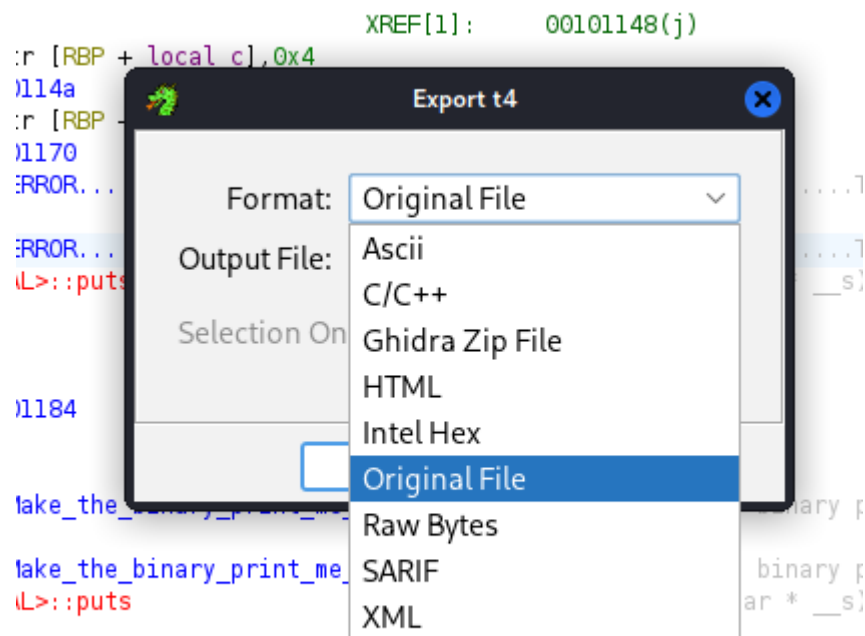
3. To Simplify Execution During Debugging

Sometimes you force a jump to skip: long loops, error handling, unwanted behaviors
You skip irrelevant paths to reach the part you want to study.

Save file by exporting program



format : original file



```

(kali@kali)-[~]
$ cd Downloads/task4

(kali@kali)-[~/Downloads/task4]
$ ./t4
zsh: permission denied: ./t4

(kali@kali)-[~/Downloads/task4]
$ chmod +x t4

(kali@kali)-[~/Downloads/task4]
$ ./t4
Make the binary print me :)

```

7. Task 5

```

main
XREF[4]: Entry Point(*),
_start:001010c4(*), 00102168(*)

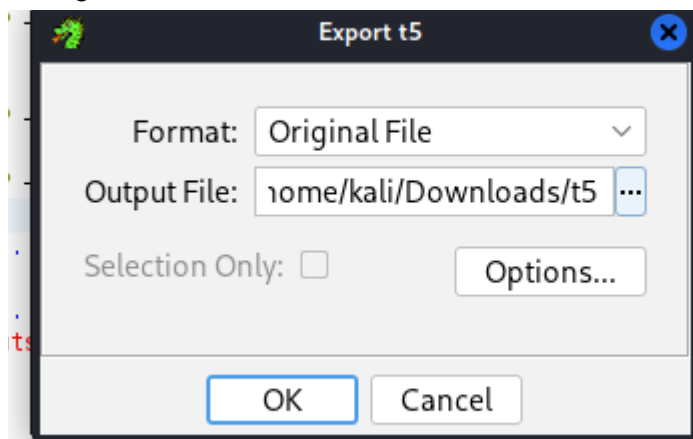
35          PUSH      RBP
18 89 e5     MOV       RBP,RSP
18 83 ec 40   SUB       RSP,0x40
39 7d cc     MOV       dword ptr [RBP + local_3c],EDI
18 89 75 c0   MOV       qword ptr [RBP + local_48],RSI
:7 45 fc     MOV       dword ptr [RBP + local_c],0x0
30 00 00 00
:b 04        JMP       LAB_001011b5

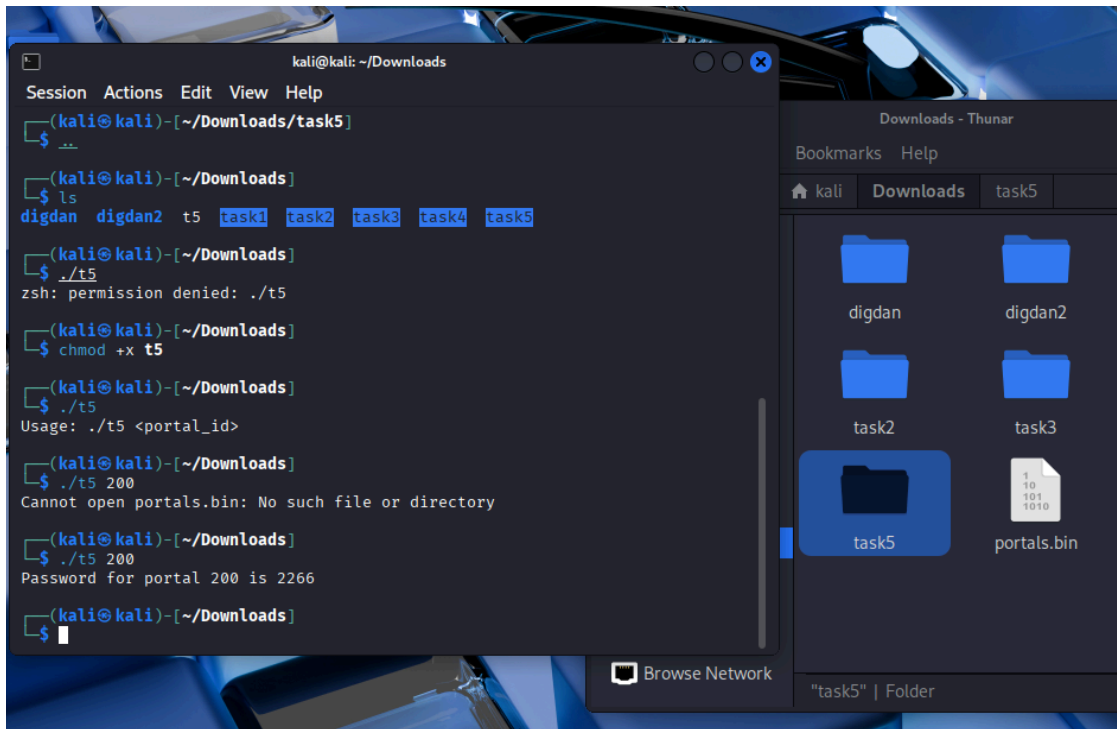
LAB_001011b1
XREF[1]: 001011b9(j)
33 45 fc 01  ADD       dword ptr [RBP + local_c],0x1

LAB_001011b5
XREF[1]: 001011af(j)
33 7d fc 04   CMP       dword ptr [RBP + local_c],0x4
7e f6        JLE       LAB_001011b1
33 7d fc 05   CMP       dword ptr [RBP + local_c],0x5
7b 19        JNP       0x001011da
18 8d 05     LEA       RAX,[s_ERROR.....The_program_has_b_00102... = "ERROR.....T
10 0e 00 00
18 89 c7     MOV       RDI=>s_ERROR.....The_program_has_b_00102... = "ERROR.....T
:b 60 fe     CALL      <EXTERNAL>::puts                                int puts(char * __s)
:f ff
:b 01 00     MOV       EAX,0x1

```

Change from **JNP** to **JMZ**





Question:

When you have a stripped binary how can you know the name of the function?

What Does “Stripped Binary” Mean?

A **stripped binary** is an executable file from which all symbolic information has been removed, including: Function names, Variable names, Debug information, Comments

This makes reverse engineering harder because instead of seeing:

```
main
check_password
decrypt
print_flag
```

you only see:

```
sub_401000
FUN_00401520
loc_7ffdf3aa
```

So you must **discover function roles manually**.

How to Identify Functions in a Stripped Binary

Even without names, the program still has:

An **entry point**

Calls to other functions

A **program flow**

These allow you to **reconstruct function meaning**.

Step 1 — Go to the Entry Point

Every binary has an **entry point**, the first instruction the program executes.

In Ghidra / IDA:

It is usually labeled as `_start` or `entry`

This function prepares the environment, then calls `main`

Example:

```
_start:  
    call main
```

If symbols are stripped, it may not be explicitly called `main`, but you still see the call.

Step 2 — Follow the Calls from the Entry Point

From `_start`, identify which function is:

Called first

Uses program arguments

Contains logic such as loops, comparisons, I/O

This main-style function often becomes your **reconstructed main**.

Example:

```
call sub_401230
```

You inspect `sub_401230` and see:

input handling
program logic
return value

So you rename it manually in the disassembler to:

`main`

Step 3 — Execute the Program to Observe Behavior

Running the binary helps you understand what each function does.

Why run it?

Because seeing *how the program behaves* helps identify:

which function prints text
which function checks input
which function handles encryption / decryption
which function is triggered for errors

Technique:

1. **Run the program** normally or inside a debugger (gdb, gef, pwndbg).
2. Notice what output or action happens.
3. Set breakpoints at functions inside Ghidra/IDA.
4. When the program hits a function, you know “this function is responsible for X.”

Step 4 — Match Observed Behavior to Assembly

Example:

If the program prints:

`Enter password:`

Then inside the binary you find a function calling `puts("Enter password:").`

Rename that function:

```
ask_password()
```

If another function checks user input:

```
cmp eax, correct_value
```

Rename it:

```
check_password()
```

You slowly rebuild meaning.

Trainer: YESSINE KALLEL Securinets ISI
Done by: SAHAR FEKI
Training Manager & External Relation Manager